Ground Robot Waypoint Navigation

Avoiding Dynamic Obstacles via MPC, IRL, and Imitation Learning

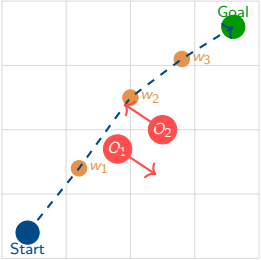
Team Markov Minds

Outline

1. Problem Overview
2. Problem Formalization
3. Method 1: Model Predictive Control (MPC)
4. Method 2: Inverse Reinforcement Learning (IRL)
5. Method 3: Behaviour Cloning & DAgger
6. Conclusion

Motivation

- Autonomous ground robots must navigate from a **start** to a **goal** through a sequence of **waypoints**
- Real-world environments contain dynamic obstacles (pedestrians, other robots, moving objects)
- Static planners fail to react in time — we need **reactive, predictive, and learnable** policies
- Three complementary approaches are studied:
  1. Model Predictive Control (MPC)
  2. Inverse Reinforcement Learning (IRL)
  3. Behaviour Cloning & DAgger



Environment

State, Action, and Environment

Robot State

$$\mathbf{s}_t = (x_t,\ y_t,\ \theta_t,\ v_t) \in \mathcal{S}$$

where (x_t, y_t) is position, θ_t is heading, v_t is speed.

Action Space

$$\mathbf{u}_t = (\omega_t,\ a_t) \in \mathcal{U}$$

where ω_t is angular velocity (steering) and a_t is linear acceleration.

Dynamic Obstacles

$$\mathcal{O}_t = \{(\mathbf{p}_t^{(i)},\ \dot{\mathbf{p}}_t^{(i)},\ r^{(i)})\}_{i=1}^{N_o}$$

Each obstacle i has position $\mathbf{p}_t^{(i)}$, velocity $\dot{\mathbf{p}}_t^{(i)}$, and radius $r^{(i)}$.

Scenario Modeling and Environment Setup

Episode Structure

- **Robot:** $\mathbf{s}_t = (x_t, y_t, \theta_t, v_t)$
- **Waypoints:** $\mathcal{W} = \{w_1, w_2, w_3\}$
- **Obstacles:** $\mathcal{O}_t = \{(\mathbf{p}_t^{(i)}, \dot{\mathbf{p}}_t^{(i)}, r^{(i)})\}_{i=1}^{2}$
- **Switching:** $\|\mathbf{p}_t - w_k\| \leq \epsilon$

Modeling Assumptions

- Circular obstacles: radius $r^{(i)}$
- Full observability of $\mathbf{p}_t^{(i)}, \dot{\mathbf{p}}_t^{(i)}$
- Prediction: $\hat{\mathbf{p}}_{t+k}^{(i)} = \mathbf{p}_t^{(i)} + k\Delta t\, \dot{\mathbf{p}}_t^{(i)}$
- Episode: $\mathbf{s}_0 \sim p(\mathbf{s}_0)$, terminate on success/collision

Scenario Diversity

Waypoints w_k, obstacle initial states $(\mathbf{p}_0^{(i)}, \dot{\mathbf{p}}_0^{(i)})$, and velocities are randomized across episodes.

Unicycle Dynamics and Waypoint Task

Unicycle Kinematic Model

$$\begin{pmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \\ \dot{v} \end{pmatrix} = \begin{pmatrix} v\cos\theta \\ v\sin\theta \\ \omega \\ a \end{pmatrix}$$

Discretized with timestep Δt: $\mathbf{s}_{t+1} = f(\mathbf{s}_t, \mathbf{u}_t)$.

Waypoint Navigation Task

Given a sequence of waypoints $\mathcal{W} = \{w_1, w_2, \ldots, w_K\}$:

- Robot must visit each w_k within tolerance ϵ in order
- Must avoid all dynamic obstacles: $\|\mathbf{p}_t - \mathbf{p}_t^{(i)}\| > r^{(i)} + r_{\text{robot}}\ \forall\, i, t$
- Respect actuation limits: $|\omega_t| \leq \omega_{\max}$, $v_{\min} \leq v_t \leq v_{\max}$

Formal Objective

Constrained Optimal Control Problem

$$\pi^* = \arg\min_{\pi} \; \mathbb{E}\left[\sum_{t=0}^{T} c(\mathbf{s}_t, \mathbf{u}_t)\right]$$

$$\text{subject to} \quad \mathbf{s}_{t+1} = f(\mathbf{s}_t, \mathbf{u}_t), \quad \mathbf{u}_t \in \mathcal{U}, \quad g(\mathbf{s}_t, \mathcal{O}_t) \leq 0$$

Cost function $c(\mathbf{s}, \mathbf{u})$:

$$c = \underbrace{\lambda_1 \|s_{xy} - w_k\|^2}_{\text{waypoint tracking}} + \underbrace{\lambda_2 \|\mathbf{u}\|^2}_{\text{control effort}} + \underbrace{\lambda_3 c_{\text{obs}}}_{\text{obstacle penalty}}$$

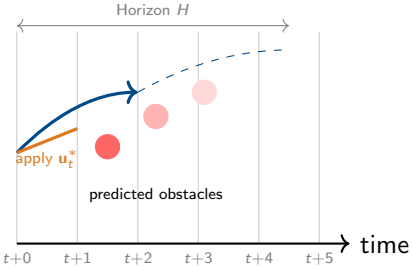
Collision constraint g:

$$g(\mathbf{s}_t, \mathcal{O}_t) = r^{(i)} + r_{\text{robot}} - \|\mathbf{p}_t - \mathbf{p}_t^{(i)}\|$$

Infeasible if $g > 0$ for any i.

MPC — Core Idea

- At each timestep t, solve a **finite-horizon optimization** over prediction horizon H
- Apply only the **first action** $\mathbf{u}_t^*$, then re-plan at $t+1$
- Incorporates **obstacle predictions** $\hat{\mathbf{p}}_{t+k}^{(i)}$ over horizon
- Handles **constraints** explicitly (safety, actuation limits)



Key Advantage

MPC acts as the **expert** / oracle for IRL and imitation learning methods.

MPC — Formulation Applied to Our Problem

At time t, solve:

$$\min_{\mathbf{u}_{t:t+H-1}} \sum_{k=0}^{H-1} \left[\lambda_1 \|\mathbf{p}_{t+k} - w_{\text{active}}\|^2 + \lambda_2 \|\mathbf{u}_{t+k}\|^2 + \lambda_3 \sum_i \max(0,\ r_{\text{safe}} - d_{t+k}^{(i)})^2 \right] + \lambda_4 \|\mathbf{p}_{t+H} - w_{\text{active}}\|^2$$

subject to:

$$\mathbf{s}_{t+k+1} = f(\mathbf{s}_{t+k}, \mathbf{u}_{t+k}), \quad \mathbf{u}_{t+k} \in \mathcal{U}, \quad d_{t+k}^{(i)} = \|\mathbf{p}_{t+k} - \hat{\mathbf{p}}_{t+k}^{(i)}\| \geq r^{(i)} + r_{\text{robot}}$$

Obstacle Prediction

Constant-velocity model:
$\hat{\mathbf{p}}_{t+k}^{(i)} = \mathbf{p}_t^{(i)} + k\Delta t \cdot \dot{\mathbf{p}}_t^{(i)}$

Waypoint Switching

Advance to w_{k+1} when $\|\mathbf{p}_t - w_k\| \leq \epsilon$

IRL — Core Idea

- MPC generates **expert demonstrations**: $\mathcal{D} = \{(\mathbf{s}_t, \mathbf{u}_t^{\text{MPC}})\}$
- IRL **recovers the reward function** $R(\mathbf{s}, \mathbf{u})$ that explains expert behavior
- A policy π_θ is then optimized under the learned reward
- Once learned, the policy runs **without** the MPC solver at test time

Reward Parameterization

Assume reward is a **linear combination** of features $\phi(x, u)$:

$$R(x, u) = w^\top \phi(x, u)$$

Features: relative position to active waypoint, relative position and velocity to Top-K nearest obstacles, control magnitude.

IRL — Maximum Entropy Objective & Gradient

Maximum Entropy IRL Objective

Optimize weights λ to maximize likelihood over expert demonstrations $\Xi = \{\tilde{\zeta}_0, \tilde{\zeta}_1, \ldots\}$:

$$\max_{\lambda} \sum_{\tilde{\zeta}_i \in \Xi} \lambda^\top f(\tilde{\zeta}_i) - \log Z(\lambda), \qquad Z(\lambda) = \int e^{\lambda^\top f(\tau)}\, d\tau$$

This is solved using a gradient ascent algorithm on λ.

Gradient of the Objective

$$\nabla_\lambda J(\lambda) = \underbrace{\sum_{\tilde{\zeta}_i \in \Xi} f(\tilde{\zeta}_i)}_{\text{empirical feature counts}} - \underbrace{\mathbb{E}_{\tau \sim p^*(\tau,\lambda)}\big[f(\tau)\big]}_{\text{expected feature counts under } p^*}$$

IRL — Algorithm

MaxEnt IRL: Iterative Reward Recovery

1. **Collect** expert demonstrations $\Xi = \{\tilde{\zeta}_0, \tilde{\zeta}_1, \ldots\}$ from MPC. Compute empirical feature counts $\hat{\mu}^* = \sum_{\tilde{\zeta}_i} f(\tilde{\zeta}_i)$.
2. **Initialize** reward weights λ.
3. **Repeat until convergence:**
   a. Compute optimal policy π_λ under current reward $R(x, u) = \lambda^\top \phi(x, u)$ (e.g. via RL).
   b. Roll out π_λ; estimate expected feature counts via Monte Carlo:
   $$\hat{\mu}(\lambda) \approx \mathbb{E}_{\tau \sim p^*(\tau, \lambda)}\big[f(\tau)\big]$$
   c. Gradient step on λ:
   $$\lambda \leftarrow \lambda + \alpha \left(\hat{\mu}^* - \hat{\mu}(\lambda)\right)$$
4. **Return** $\hat{w}^* = \lambda$.

IRL — Policy Optimization

Policy Optimization under Learned Reward

With the recovered reward weights $\hat{w}^* = \lambda$, fix $R(x, u) = \hat{w}^{*\top}\phi(x, u)$ and train π_θ via any RL algorithm:

$$\max_{\theta} \; \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{T-1} \gamma^t R(x_t, u_t)\right]$$

Key Properties

- Reward $\hat{w}^*$ is **fixed** after IRL convergence — policy training is standard RL.
- Policy π_θ is a **feedforward network**: no solver needed at test time.
- Generalizes to obstacle configurations **unseen during IRL**, since the reward encodes intent (waypoint progress, obstacle avoidance) rather than specific trajectories.

Behaviour Cloning: Core Idea & Distribution Shift

1. Supervised Imitation (The Mechanism)

We train policy π_θ by minimizing the Mean Squared Error against the expert:

$$\min_\theta \frac{1}{|\mathcal{D}|} \sum_{(\mathbf{s},\mathbf{u}^*)\in\mathcal{D}} \|\pi_\theta(\mathbf{s}) - \mathbf{u}^*\|^2$$

2. Flaw: Covariate Shift

Initial error ϵ leads to unseen states ($P_{test} \neq P_{train}$). Errors compound because actions determine future inputs, yielding a quadratic bound:

$$\sum_{t=1}^{T} \epsilon t = \epsilon \frac{T(T+1)}{2} \implies \mathbb{E}[\text{Total Error}] = O(\epsilon T^2)$$

DAgger: Training, Validation & The Best Policy

DAgger Algorithm [Ross et al., 2011]

```
Initialize dataset D = ∅, policy π_1
for iteration n = 1, 2, ..., N do
    Let π_mix = β_n π* + (1 − β_n) π_n
    Roll out π_mix, collect uncertain states S_n
    Query Expert for true labels:
  D_n = {(s, π*(s)) : s ∈ S_n}
    Aggregate: D ← D ∪ D_n
    Train/Val Split D (e.g., 80/20 split)
    Retrain π_{n+1} on D using early stopping
    Test π_{n+1} in pure simulation (no expert)
end for
Return best π_i based on test performance
```

The Core Intuition

When the Learner drives, it inevitably visits **uncertain, off-center states**. We pause, ask the MPC how to recover, and add those exact lessons to $\mathcal{D}$.

Preventing Failure

- **Mathematical Validation:** We split the dataset to monitor validation loss, preventing the neural network from memorizing data .
- **Unassisted Simulation Testing:** We evaluate every iteration in a pure simulation without expert intervention. Then deploy best iteration.

The Theoretical Guarantee ($\mathbf{P_{train}} \approx \mathbf{P_{test}}$)

Because DAgger actively trains the network on its own natural mistakes, the training and testing distributions align. **Result:** Errors no longer compound quadratically. The policy error drops to a highly stable, linear bound of $\mathbf{O}(\epsilon \mathbf{T})$.

Architecture: Implementing DAgger for Navigation

1. The Robot's Eyes (State $\mathbf{s}$)

The neural network takes a flattened numerical array representing the physical simulation state:

$$\mathbf{s} = \left[\mathbf{p}_{\text{rel}},\ v,\ \Delta w_k,\ \{d^{(i)}, \dot{\mathbf{p}}_{\text{rel}}^{(i)}\}_{i=1}^{K_o}\right]$$

2. The AI's Brain (Policy π_θ)

Network Architecture: Feedforward MLP
Mapping: $\mathbf{s} \to (\omega, a)$

- ω: Angular velocity (yaw/steering).
- a: Linear acceleration (throttle).

3. The PyBullet Training Pipeline

1. Rollout π_n
Collect uncertain $\mathcal{S}_n$

→

2. Query & Agg.
$\mathcal{D} \leftarrow \mathcal{D} \cup \mathcal{D}_n$

→

3. Retrain
Split & Update π_{n+1}

→

4. Evaluate
Unassisted Test

Deploy surviving weights to next iteration

Evaluation Metrics

Navigation Performance

- **Waypoint success rate**: fraction of w_k reached within ϵ
- **Goal success rate**: fraction of full episodes completed
- **Path length**: total distance travelled vs. optimal
- **Time to goal**: episode duration

Safety & Efficiency

- **Collision rate**: % episodes with obstacle contact
- **Minimum clearance**: $\min_t d_t^{(i)} - r_{\text{safe}}$
- **Inference time**: ms per step
- **Expert query count** (DAgger only)

Baseline Note

MPC is evaluated as the **reference baseline**. Due to solver heuristics and finite horizons, it is not strictly a theoretical upper bound, but IRL/DAgger are evaluated relative to its performance.

Summary

1. **Problem**: Navigating waypoints among dynamic obstacles formalized as constrained optimal control.
2. **MPC**: Receding-horizon optimizer; serves as the robust expert oracle.
3. **IRL**: Recovers the latent trajectory reward from MPC; deploys via RL without a solver.
4. **DAgger**: Iteratively queries MPC at learner-visited states to eliminate distribution shift with an $O(\epsilon T)$ regret bound.

Key Takeaway

IRL and DAgger **distill** the expensive MPC solver into a **lightweight neural policy** that generalises well.

References

Ziebart et al., *MaxEnt IRL*, AAAI 2008.
Ross et al., *DAgger*, AISTATS 2011.

Thank You

Questions?

Ground Robot Waypoint Navigation
Avoiding Dynamic Obstacles